# Mini Project: Cohort 21: Affordable Housing Price Estimator

## Date: 02.03.2026

Today, our team conducted a meeting to discuss the progress of the mini project, focusing mainly on **model building, model evaluation, and the user interface development**.

During the meeting, the model development team explained the algorithms they selected and how the model was trained using the preprocessed dataset. They discussed why a particular algorithm was chosen and how the training process was carried out. We also reviewed the initial results generated by the model.

Next, we discussed the **evaluation of the model**. The team presented performance metrics such as accuracy and other evaluation measures to understand how well the model is performing. We analyzed whether the model needs further improvement, tuning, or optimization to enhance its performance.

The UI development team then demonstrated the progress of the user interface. They explained how the front-end is designed to interact with the trained model and how users will input data and view predictions. Suggestions were shared to improve usability, layout, and overall user experience.

Overall, the meeting was productive, as we were able to understand the progress made by different teams, evaluate the model's performance, and align our next steps toward completing the project successfully.

## Group Discussion: Day 3

## Model Building and Evaluation:

The first step in creating a model is to define the problem as a supervised regression task, with the goal being to use two independent variables—total_sqft and Distance_to_nearest_MRT_station_m—to predict the continuous target variable price. These two characteristics are chosen as the explanatory variables because the objective is to comprehend how housing size and proximity to public transport affect property prices. First, the dataset is separated into testing and training sets. Because the model must learn patterns from the training data and then be assessed on test data that hasn't been seen before, this separation is crucial because it gauges the model's capacity for generalisation rather than memorisation.

RobustScaler is used to scale features prior to training the Linear Regression model. Outliers are frequent in housing datasets because of opulent homes or abnormally high square footage values. The mean and standard deviation, which are sensitive to extreme values, are the foundation of conventional scaling techniques like standardisation. The mean can be greatly shifted and the scaling process distorted if there are a few very large properties or very expensive homes. RobustScaler gets around this restriction by transforming data using the median and interquartile range (IQR). The scaled features more accurately depict the data's central distribution because the median and IQR are resistant to extreme values. This enhances numerical stability during training and guarantees that extreme observations do not disproportionately affect the regression model.

After building our machine learning model, we evaluated its performance to understand how well it predicts house prices. We divided the dataset into training and testing sets so that the model could be trained on one portion of the data and tested on unseen data. This helps us check whether the model is learning properly or just memorizing the data.

To measure accuracy, we used evaluation metrics such as $R^2$ Score and Mean Absolute Error (MAE). The $R^2$ score helped us understand how much variation in house prices is explained by our model, while MAE showed the average difference between the predicted and actual prices.

The evaluation results indicated that our model performs reasonably well and can predict housing prices with good accuracy. This confirms that the preprocessing and feature selection steps were effective, making the model reliable for our Affordable Housing Price Estimator project under SDG 11 – Sustainable Cities and Communities.

## UI of the Model

The frontend of our project was developed using Streamlit, a Python framework that allows us to build simple and interactive web applications for machine learning models. It helps convert our trained model into a user-friendly interface that can run directly in the browser.

We imported essential libraries such as streamlit, pandas, numpy, and pickle. The trained Linear Regression model (house_model.pkl) was loaded using pickle.load() so that it could be used for real-time predictions.

The interface includes input fields created using st.number_input() for numerical values like square feet, BHK, and bathrooms, and st.selectbox() for selecting location. These fields allow users to enter property details easily.

When the user clicks the "Predict Price" button, the entered inputs are converted into the required numerical format and passed to the trained model using model.predict(). The predicted house price is then displayed instantly on the screen using st.success().

Overall, the Streamlit frontend makes our Affordable Housing Price Estimator interactive, simple, and easy for users to access and understand predictions in real time.